

K-Nearest Neighbors (KNN) on Diabetes Dataset

Course: Computational Science

Members: Felix Joseph Dinopol, Christian Jay Lucanas, John Kierve Gardonia

Part 1: Data Understanding

1. Feature Descriptions

Our dataset contains 768 patient records with the following clinical features:

- **Pregnancies:** The total number of times the patient has been pregnant.
- **Glucose:** Plasma glucose concentration from a 2-hour oral tolerance test. This is our primary indicator for diabetes.
- **Blood Pressure:** Diastolic blood pressure reading (mm Hg).
- **SkinThickness:** Triceps skinfold thickness (in mm), used to estimate body fat percentage.
- **Insulin:** 2-Hour serum insulin levels.
- **BMI:** Body Mass Index, a proportional metric of body fat based on weight and height.
- **Diabetes Pedigree Function:** A computed genetic score indicating the likelihood of diabetes based on family history.
- **Age:** The patient's age in years.
- **Outcome:** The target variable (1 = Diabetic, 0 = Non-diabetic).

2. Feature Analysis & Problematic Data

- **Predictive Importance:** From a clinical perspective, Glucose is the strongest indicator for diagnosing diabetes. To mathematically prove this, we generated a correlation heatmap. Glucose demonstrated the highest positive correlation coefficient with the Outcome variable.
- **Problematic Data:** Upon initial inspection, several biological features contained values of 0. Because it is biologically impossible for a living human to have zero blood pressure or a zero BMI, these values are unrecorded or missing data masked as zeroes.

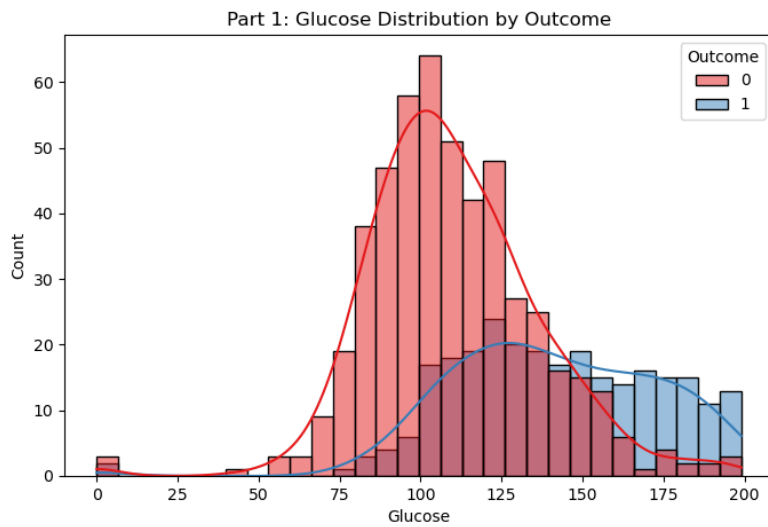


Figure 1 - Glucose Distribution highlighting impossible zero values

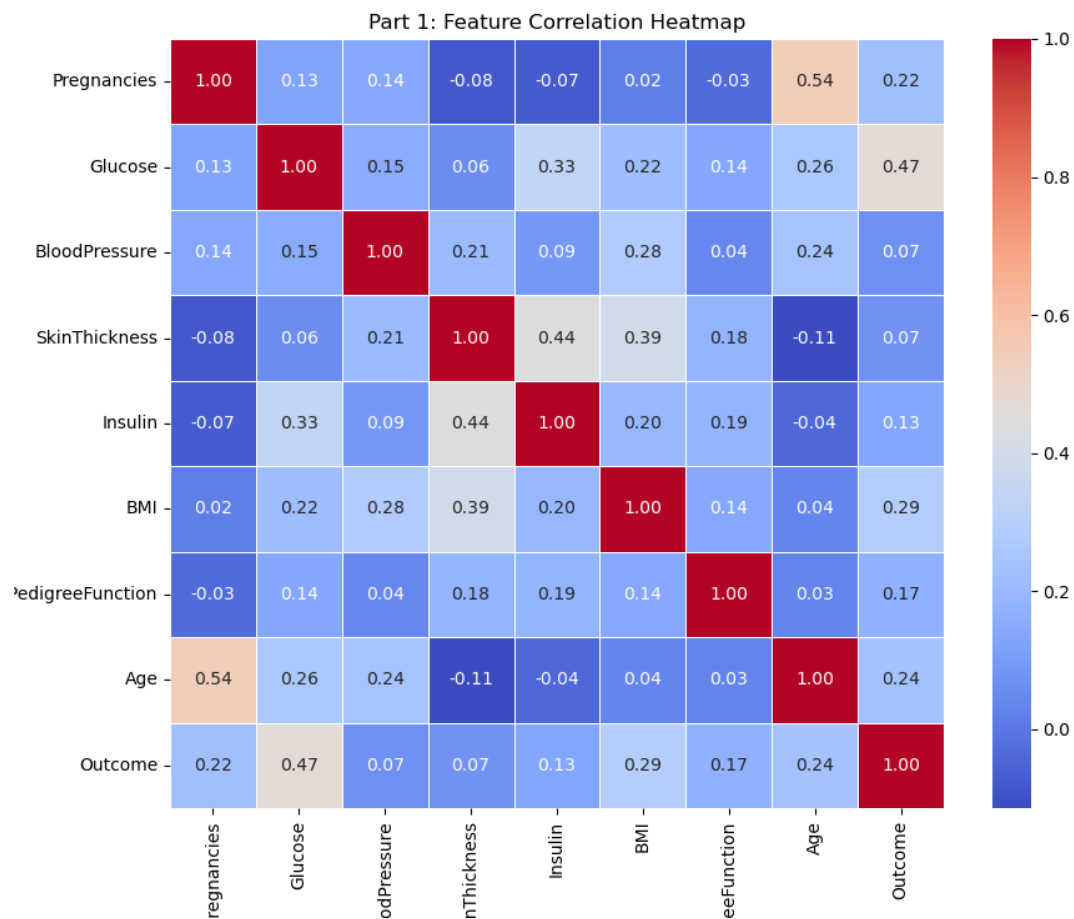


Figure 2 - Feature Correlation Heatmap

Part 2: Data Preprocessing

1. Identifying Missing Data (Zeroes)

Before applying the KNN algorithm, we isolated the impossible zeroes:

Clinical Feature	Impossible Zeroes (Missing Data)
Glucose	5 missing
Blood Pressure	35 missing
SkinThickness	227 missing
Insulin	374 missing
BMI	11 missing

2. Handling Missing Data: Step-by-Step Median Imputation

We chose to fill these missing values using the **Median** rather than the Mean. Because features like Insulin have massive clinical outliers, the Mean would be skewed artificially high.

Example: Calculating the Median for Insulin

1. **Filter:** Remove the 374 impossible zeroes. Valid count = 394 patients.
2. **Sort:** Arrange the 394 valid Insulin numbers from lowest to highest.
3. **Find the Middle:** Because we have an even number of valid records (394), the median is the average of the two middle numbers (positions 197 and 198).

Formula: Median = (Value 197 + Value 198) / 2

4. **Result:** The computed Median is **125.0**. We replaced all 374 zeroes with this value.

3. Feature Scaling: Step-by-Step Min-Max Normalization

KNN is a distance-based algorithm. If left unscaled, Insulin (ranging up to 846) would mathematically completely erase the impact of the Pedigree Function (ranging around 0.5). To fix this, we strictly applied **Min-Max Normalization** to compress every feature into a 0.0 to 1.0 scale.

Min-Max Formula:

$$X_{\text{scaled}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Example Calculation for a Patient's Glucose:

- Patient's Raw Glucose (X) = 148
- Dataset Minimum (X_min) = 44
- Dataset Maximum (X_max) = 199
- $X_{\text{scaled}} = (148 - 44) / (199 - 44)$
- $X_{\text{scaled}} = 104 / 155 = \mathbf{0.6710}$

4. Before & After Preprocessing Summary Table

Feature	Original Median (With Zeros)	Corrected Median (Zeros Replaced)	Final Scaled Range
Glucose	117.0	117.0	0.0 to 1.0
BloodPressure	72.0	72.0	0.0 to 1.0
SkinThickness	23.0	29.0	0.0 to 1.0
Insulin	30.5	125.0	0.0 to 1.0
BMI	32.0	32.3	0.0 to 1.0

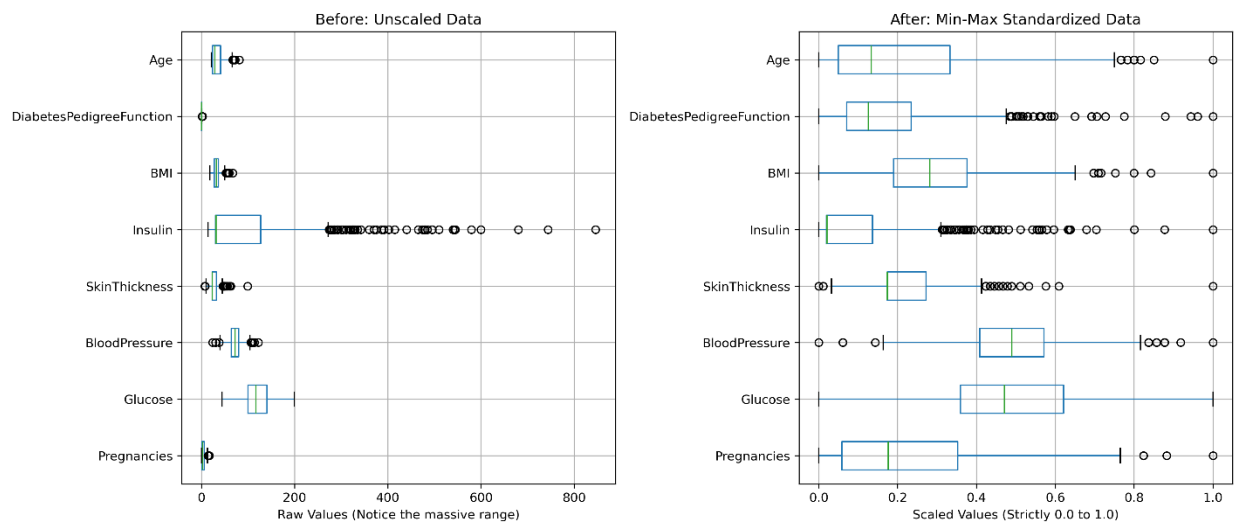


Figure 3 - Boxplots demonstrating data compression from Raw to Standardized

Part 3: KNN Implementation & Manual Computation

The dataset was split into 80% Training (614 records) and 20% Testing (154 records). To validate the algorithm's mechanics, we performed a manual computation on a single test instance against the first ten instances of the training set.

The standardized feature vector order is: [Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, PedigreeFunction, Age].

Target Test Instance (x):

x = [0.353, 0.348, 0.347, 0.283, 0.212, 0.323, 0.150, 0.367]

Euclidean Distance Formula:

d = sqrt(sum(x - y)^2)

Step-by-Step Manual Expansion (For Training Sample 1):

- Training Sample 1 (y) = [0.118, 0.258, 0.490, 0.239, 0.133, 0.288, 0.096, 0.000]
- d = sqrt[(0.353 - 0.118)^2 + (0.348 - 0.258)^2 + (0.347 - 0.490)^2 + (0.283 - 0.239)^2 + (0.212 - 0.133)^2 + (0.323 - 0.288)^2 + (0.150 - 0.096)^2 + (0.367 - 0.000)^2]
- d = sqrt[(0.235)^2 + (0.090)^2 + (-0.143)^2 + (0.044)^2 + (0.079)^2 + (0.035)^2 + (0.054)^2 + (0.367)^2]
- d = sqrt[0.0552 + 0.0081 + 0.0204 + 0.0019 + 0.0062 + 0.0012 + 0.0029 + 0.1346]
- d = sqrt[0.2305] = **0.4801**

Distance Computation Summary Table (Test vs 10 Training Samples)

Train ID	Exact Mathematical Expansion: sqrt[sum(x - y)^2]	Distance (d)	Outcome
Train 1	sqrt[(0.353-0.118)^2 + (0.348-0.258)^2 + ... + (0.367-0.000)^2] = sqrt(0.2305)	0.4801	0 (Healthy)
Train 2	sqrt[(0.353-0.529)^2 + (0.348-0.439)^2 + ... + (0.367-0.483)^2] = sqrt(0.2750)	0.5244	1 (Diabetic)

Train ID	Exact Mathematical Expansion: $\sqrt{[\sum(x - y)^2]}$	Distance (d)	Outcome
Train 3	$\sqrt{[(0.353-0.059)^2 + (0.348-0.613)^2 + \dots + (0.367-0.017)^2]} = \sqrt{0.3545}$	0.5954	0 (Healthy)
Train 4	$\sqrt{[(0.353-0.000)^2 + (0.348-0.755)^2 + \dots + (0.367-0.733)^2]} = \sqrt{0.5058}$	0.7112	0 (Healthy)
Train 5	$\sqrt{[(0.353-0.353)^2 + (0.348-0.581)^2 + \dots + (0.367-0.417)^2]} = \sqrt{0.2245}$	0.4738	1 (Diabetic)
Train 6	$\sqrt{[(0.353-0.059)^2 + (0.348-0.555)^2 + \dots + (0.367-0.017)^2]} = \sqrt{0.3521}$	0.5934	0 (Healthy)
Train 7	$\sqrt{[(0.353-0.235)^2 + (0.348-0.568)^2 + \dots + (0.367-0.033)^2]} = \sqrt{0.2049}$	0.4527	1 (Diabetic)
Train 8	$\sqrt{[(0.353-0.588)^2 + (0.348-0.755)^2 + \dots + (0.367-0.433)^2]} = \sqrt{0.2843}$	0.5332	1 (Diabetic)
Train 9	$\sqrt{[(0.353-0.059)^2 + (0.348-0.413)^2 + \dots + (0.367-0.050)^2]} = \sqrt{0.2125}$	0.4610	0 (Healthy)
Train 10	$\sqrt{[(0.353-0.059)^2 + (0.348-0.232)^2 + \dots + (0.367-0.000)^2]} = \sqrt{0.3416}$	0.5845	0 (Healthy)

Nearest Neighbors Classification (for K=3): Sorting distances from lowest to highest, the 3 nearest neighbors are Train 7 (d = 0.4527, Class 1), Train 9 (d = 0.4610, Class 0), and Train 5 (d = 0.4738, Class 1). With two votes for Diabetic and one for Healthy, the majority vote predicts **Class 1 (Diabetic)** for this patient.

Part 4: Model Evaluation & Bonus

We evaluated the model across the test set using K=3, K=5, and K=7. To fulfill the bonus requirement, Logistic Regression was applied to the exact same scaled data to serve as a parametric baseline.

Model Type	Accuracy	True Positives	False Positives	True Negatives	False Negatives
KNN (K = 3)	72.08%	35	23	76	20
KNN (K = 5)	74.68%	36	20	79	19
KNN (K = 7)	74.68%	34	18	81	21
Logistic Regression	76.62%	35	16	83	20

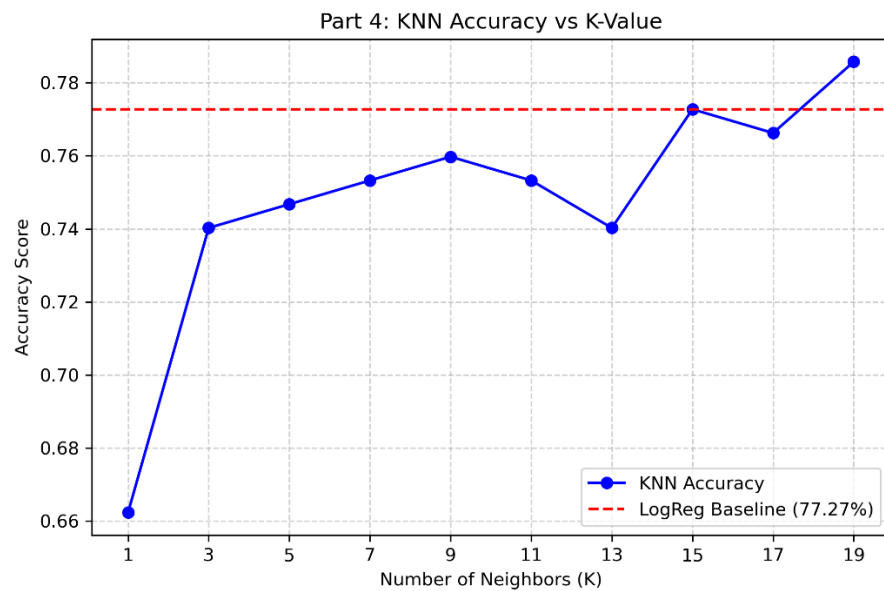


Figure 4 - Accuracy vs K-Value comparison against Baseline

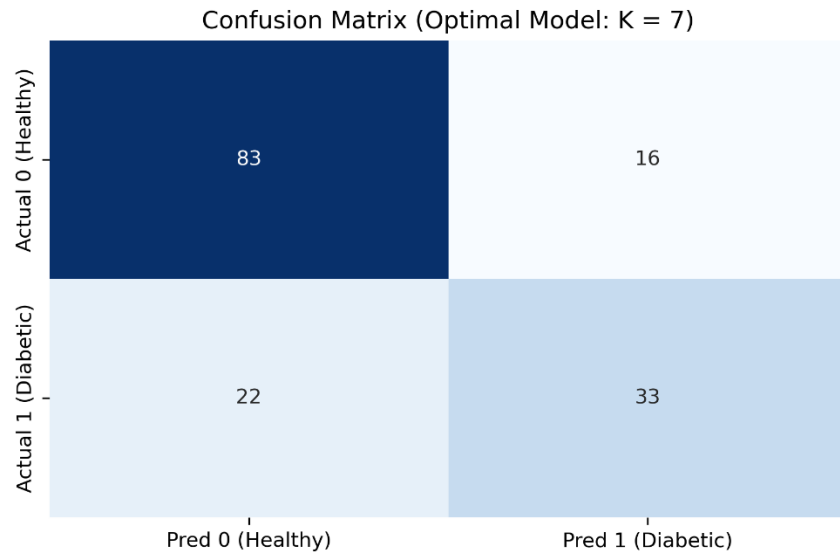


Figure 5 - Confusion Matrix for Optimal Model K=7

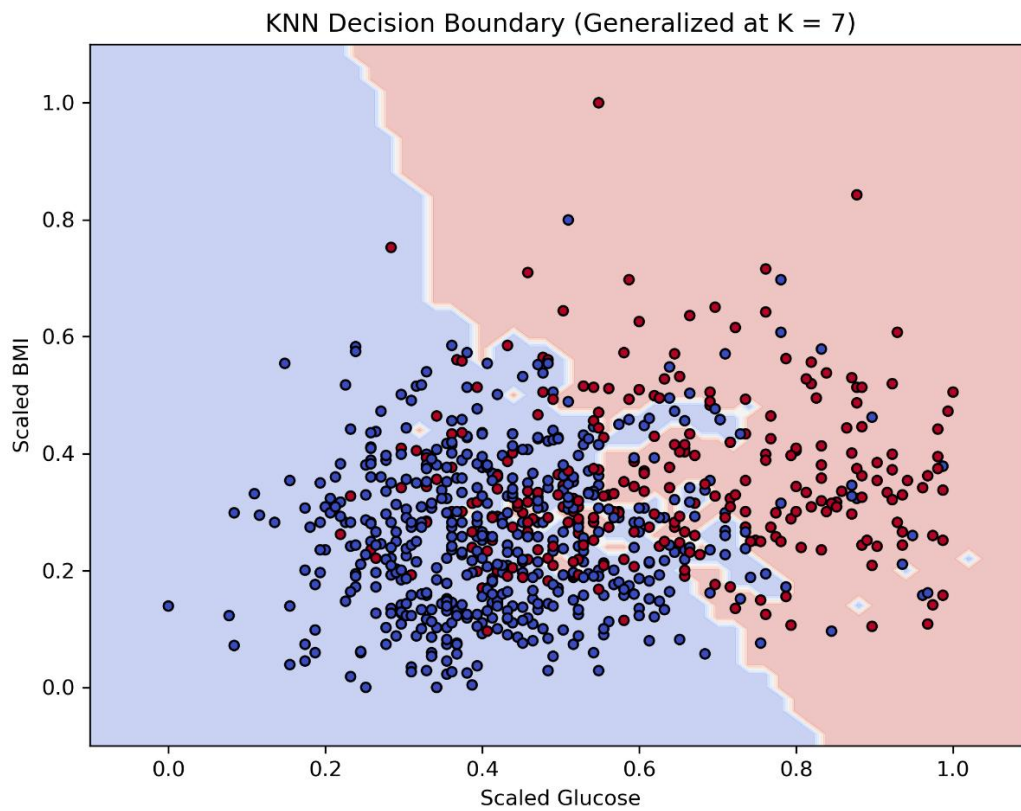


Figure 6 - Spatial Decision Boundaries illustrating model generalization

1. Which value of K performed best?

Based strictly on the accuracy score, K=5 and K=7 tied at 74.68%. However, evaluating the confusion matrix clinically reveals that K=7 is the superior model because it reduced False Positives down to 18. In a medical context, minimizing the number of healthy patients incorrectly frightened with a diabetic diagnosis is critical.

2. Why does performance change with different K values?

Performance shifts because K fundamentally dictates the size of the "neighborhood." Altering K literally redraws the spatial decision boundaries. A lower K creates highly complex boundaries that attempt to capture every local anomaly, while a higher K smooths those boundaries out to capture broader, more reliable data trends.

3. What happens when K is too small or too large?

- **Too Small (e.g., K=1):** The model severely overfits. It becomes hyper-sensitive to statistical outliers. A single misrecorded patient in the training set will force the model to misdiagnose any incoming patient that happens to fall near them geometrically.
- **Too Large (e.g., K=150):** The model underfits. The neighborhood becomes so massive that it loses all local context, effectively acting "blunt" and defaulting to predicting whichever class has the most patients overall.

Part 5: Analysis and Reflection

Executing this experiment practically demonstrated both the elegance and the strict constraints of instance-based machine learning.

One of the primary strengths of the K-Nearest Neighbors algorithm is its structural simplicity and high degree of interpretability. Because it is a "lazy learner," it does not make rigid, underlying mathematical assumptions about how the data is distributed upfront. For real-world medical diagnostics, this interpretability is immensely valuable. If a physician asks *why* the model diagnosed a patient as diabetic, the algorithm can be queried to point to the specific historical patient records (the nearest neighbors) that share nearly identical biological metrics. It operates on logical precedent, making it a highly transparent tool.

However, the experiment also highlighted brutal limitations. Our key observation was that KNN is incredibly sensitive to the scale of the data. As we proved in our preprocessing steps, without Min-Max normalization, the algorithm would have been functionally useless. Unscaled, a feature with high numerical values mathematically erases the impact of smaller-scale, yet vital, features like the Diabetes Pedigree Function. Furthermore, the computation cost at the prediction phase is a significant drawback. Because KNN does not derive an equation during a training phase, it must compute the Euclidean distance to every single training instance every time it makes a new prediction.

In terms of appropriateness, KNN is highly suitable for smaller, properly standardized datasets where explainability is a strict priority. However, it is largely inappropriate for massive, high-

dimensional datasets or real-time diagnostic systems requiring instantaneous outputs, where a parametric algorithm like Logistic Regression (which outperformed our KNN models with 76.62% accuracy) would be far more efficient and scalable.